DALIL TRAINING ACADEMY · FULL STACK WEB DEVELOPMENT

مشروع Expense Tracker

Roadmap الطالب: من الـ Backend إلى Full Stack بـ 11 يوم

المدة	نمط الشغل	عدد المراحل	التسليم
11 يوم	**فردي**	**4 مراحل**	**Zip + Demo**

عن المشروع

رح تبنوا تطبيق ويب لتتبع مصاريفكم الشخصية. بتضيفوا مصروف (العنوان، المبلغ، التصنيف، التاريخ)، وبتشوفوا كل مصاريفكم بجدول، وبتفلتروها حسب التصنيف، وبيطلعلكم الإجمالي والعدد وأعلى مصروف مباشرة. البيانات بتتخزن بـ PostgreSQL عن طريق backend مبني بـ Node.js و Express، وبيحكي معه الـ front-end عن طريق API.

شو اللي بتعرفوه، وشو الجديد؟

- **اللي أخدتوه:** HTML و CSS و JavaScript (arrays و objects و DOM و events) و Bootstrap، واستعلامات PostgreSQL (SELECT و INSERT و UPDATE و DELETE). هاد كله بيكفي لبناء الـ backend والـ front-end كامل.
- **الجديد:** JSON، و Promises و async/await و fetch، وأول تجربة مع Node.js و Express، وربط Node بقاعدة بيانات PostgreSQL. هدول ما رح ينشرحوا بمحاضرة، بتتعلموهم بأنفسكم بـ Phase 0.

شو الهدف من المشروع؟

إنكم تطبقوا اللي تعلمتوه على مشروع كامل، وتتعلموا أدوات جديدة بشكل مستقل، وتفهموا كيف الـ front-end بيحكي مع الـ backend والـ database. وهو كمان بيساعدنا نعرف وين وصلتوا، وين لازم نركز معكم بالمراحل الجاي من البرنامج.

كيف نمشي بالمشروع

- المشروع 4 مراحل (من Phase 0 لـ Phase 3). كل مرحلة إلها هدف، ومطلوب، وChecklist بعنوان «خلصت لما...».
- **هالمرة بنبني الـ backend قبل الـ front-end.** بعد Phase 0 بتبنوا الـ API كامل، وبعدين بتبنوا الواجهة مربوطة فيه مباشرة من أول يوم.
- ما تنتقلوا لمرحلة جديدة قبل ما تخلصوا الـ Checklist تبع اللي قبلها.
- في **Checkpoint واحد رسمي**: نهاية يوم 9، لما يكون المشروع كامل شغال. والتسليم النهائي يوم 11.
- يوم 10 مرن: مين متأخر بيلحق، ومين خلّص بدري بيبدأ بالـ Bonus.
- **علقتوا بالـ backend؟** اسألوا بالـ Discord بنفس قاعدة الـ 30 دقيقة. ما في مسار بديل، فكملوا واطلبوا مساعدة بدل ما توقفوا.

الجدول الزمني

اليوم	المرحلة	الشغل	التسليم
1	Phase 0	Setup البيئة، وقراءة JSON و fetch/async والـ backend و REST، ومهمة 1	—
2	Phase 0	قراءة Node/Express و Node مع PostgreSQL، ومهمتين 2 و 3	مهام Phase 0
3	Phase 1	إنشاء الـ database وملف `.env`، وربط Node بـ PostgreSQL، وأول endpoints (GET و POST)	—
4	Phase 1	باقي الـ endpoints (PUT و DELETE) والـ validation	—
5	Phase 1	اختبار كل الـ endpoints بـ Thunder Client	صور اختبار الـ API
6	Phase 2	هيكل الصفحة بـ Bootstrap، وجلب البيانات وعرضها بالجدول بـ fetch مباشرة	—
7	Phase 2	form الإضافة مع validation وربطه بـ POST، والحذف بـ DELETE	—
8	Phase 2	الفلترة، والتعديل بـ modal مربوط بـ PUT	—
9	Phase 2	spinner و alerts و try/catch لكل الحالات	**Checkpoint**
10	Buffer	تعويض المتأخر وإنهاء الربط	—
11	Phase 3	Responsive و CSS Grid و Bonus و README، وتسليم و demo	**التسليم النهائي**

الأيام تقديرية. إذا خلصتوا مرحلة بدري، كملوا على اللي بعدها.

تجهيز البيئة (أول ساعة من يوم 1)

- **Node.js** (نسخة LTS) من الموقع الرسمي nodejs.org.
- **VS Code** مع extension اسمها Live Server لتشغيل الـ front-end.
- extension: **Thunder Client** داخل VS Code لاختبار الـ API. بتدوّروا عليها من تبويب Extensions وبتثبتوها.
- **PostgreSQL** مع pgAdmin (منصّبين عندكم من كورس الـ database). تأكدوا إنه شغّال، وإنكم بتقدروا تفتحوا pgAdmin وتتصلوا فيه بكلمة السر تبعكم.

للتأكد إنه Node اشتغل، افتحوا الـ terminal واكتبوا:

```
node -v
npm -v
```

لازم يطلعلكم رقم النسخة لكل أمر. إذا طلع خطأ، صوّروا الشاشة واكتبوا بالـ Discord.

هيكل المشروع وشكل البيانات

رح تستلموا الهيكل جاهز (starter)، ومعه ملف `schema.sql` فيه جدول الطلبات وبيانات تجريبية. الـ frontend والـ backend بمجلدين منفصلين لأنهم فعليًا تطبيقين بيحكوا مع بعض:

```
expense-tracker/
├── frontend/
│   ├── index.html
│   ├── css/style.css
│   └── js/app.js
├── backend/
│   ├── server.js
│   ├── package.json
│   ├── schema.sql
│   └── .env.example
└── README.md
```

شكل المصروف (JSON)

```
{
  "id": 1,
  "title": "Lunch",
  "amount": 4.5,
  "category": "Food",
  "date": "2026-01-15"
}
```

التصنيفات المسموحة: `Food` و `Transport` و `Bills` و `Entertainment` و `Other`. بالـ database المصروف هو صف بجدول `expenses` بنفس الأعمدة. الـ `id` بيتولّد من الـ database.

المراحل بالتفصيل

Phase 0: التعلم الذاتي — يوم 1 – 2

الهدف: تتعلموا الأدوات الجديدة قبل ما تحتاجوها. رح نعطيكم ملف منفصل فيه مصادر مختارة لكل موضوع، فما تضيعوا وقت بالبحث.

المواضيع

- **JSON:** شو هو، و `JSON.stringify` و `JSON.parse`، وليش هي الصيغة اللي بيحكي فيها الـ front-end مع الـ backend.
- **Promises و async/await و fetch:** طلبات GET و POST.
- **الـ backend والـ REST:** شو الـ request والـ response، والـ status codes الأساسية (200 و 201 و 400 و 404 و 500)، وأفعال GET و POST و PUT و DELETE.
- **Node.js و npm و Express:** أول route بيرجع JSON، وليش نحتاج CORS.
- **Node.js و PostgreSQL:** مكتبة `pg`، وإرسال الاستعلامات بـ parameters (`$1` و `$2`)، وملف `.env`.

3 مهام صغيرة للتأكد إنكم فهمتوا

المهمة	المطلوب	كيف بتعرفوا إنها خلصت
مهمة 1	اجلبوا بيانات من public API بـ `fetch` واعرضوها بـ Bootstrap cards	الـ cards بتظهر بالصفحة من بيانات حقيقية
مهمة 2	اعملوا Express server فيه route بيرجع JSON	بتشوفوا الـ JSON من المتصفح ومن Thunder Client
مهمة 3	وصّلوا Express بـ PostgreSQL: route بيقرأ جدول تجريبي بـ `pg` وبـ parameters	بتشوفوا بيانات الجدول بـ Thunder Client، وأي غلط ما بيوقّع الـ server `id`

Checklist: خلصت لما...

- ☐ حلّيت المهام الثلاث الصغيرة وكلها شغالة.
- ☐ بقدر أشرح بكلماتي الفرق بين GET و POST، وشو يعني status `404`.
- ☐ بعرف ليش الـ front-end بيحتاج CORS ليحكي مع الـ server.
- ☐ بعرف ليش بنبعت القيم كـ parameters (`$1`) بدل ما ندمجها بنص الـ SQL.
- ☐ **سلّمت** مجلد فيه المهام الثلاث (يوم 2).

Phase 1: بناء الـ Backend كامل — يوم 3 – 5

الهدف: تبنوا API كامل بيدير المصاريف وبيخزنها بـ PostgreSQL، وتختبروه لحاله كامل قبل ما تكتبوا ولا سطر HTML.

تجهيز الـ database (أول ساعة من يوم 3)

- أنشئوا database فاضية اسمها `expense_tracker` من pgAdmin.
- شغّلوا عليها ملف `schema.sql`. هو بيعمل جدول `expenses` وبيضيف بيانات تجريبية.
- انسخوا `.env.example` لملف جديد اسمه `.env` واكتبوا فيه كلمة سر PostgreSQL. **ملف `.env` ما بينرفع ولا بيتسلّم أبدًا.**
- ثبّتوا الـ packages: `npm install express cors pg dotenv`.

الـ Endpoints المطلوبة

Method	Path	الوظيفة	نجاح	أخطاء
GET	/api/expenses	ترجع كل المصاريف	200	–
GET	/api/expenses/:id	ترجع مصروف واحد	200	404
POST	/api/expenses	إضافة مصروف جديد	201	400
PUT	/api/expenses/:id	تعديل مصروف	200	400, 404
DELETE	/api/expenses/:id	حذف مصروف	200	404

كيف بتجربوا endpoint بـ Thunder Client

- **شغّلوا الـ server** من الـ terminal، وخلوه شغال.
- **افتحوا Thunder Client** من أيقونته بالشريط الجانبي بـ VS Code، واضغطوا **New Request**.
- **اختاروا الـ Method** (GET أو POST...) واكتبوا الـ URL، مثلًا `http://localhost:3000/api/expenses`.
- **للـ POST و PUT:** من تبويب **Body** اختاروا **JSON** واكتبوا بيانات المصروف.
- **اضغطوا Send**، وبتشوفوا الـ status code والـ JSON اللي رجع بجانب الـ request.

قواعد الشغل

- **كل الاستعلامات بـ parameters:** `pool.query("... WHERE id = $1", [id])`. ممنوع تدمجوا بيانات المستخدم بنص الـ SQL، لأنه هيك بيصير SQL injection.
- الـ database هي اللي بتولّد الـ `id`. استخدموا `RETURNING` عشان ترجّعوا الصف الجديد.
- انتبهوا: مكتبة `pg` بترجّع `NUMERIC` كنص و`DATE` كـ Date (وممكن يتزحزح اليوم). حلّوا الاثنين بالـ SELECT (بتلاقوا تلميح بالـ starter).
- الـ validation بالـ server: إذا البيانات ناقصة أو غلط ارجعوا `400` مع JSON فيه `message` بيشرح المشكلة. التصنيف لازم يكون وحد من الخمسة المسموحة، والمبلغ رقم أكبر من صفر. تأكدوا من الـ `id` قبل الاستعلام، وإذا مش موجود أو غلط ارجعوا `404`.
- فعّلوا CORS بحيث الـ front-end يقدر يحكي مع الـ server.
- اختبروا كل endpoint بـ Thunder Client **قبل** ما تبدأوا بالـ front-end بـ Phase 2.

Checklist: خلصت لما...

- ☐ الـ 5 endpoints كلها بتشتغل وبترجع الـ status code الصح.
- ☐ إذا بعتّ بيانات غلط بيرجع `400` مع رسالة مفهومة.
- ☐ بعد ما أطفّي الـ server وأشغّله من جديد، البيانات لسا موجودة (بالـ database).
- ☐ كل الاستعلامات بـ `$1` و `$2`، ولا في نص SQL مدموج ببيانات من المستخدم.
- ☐ التاريخ بيرجع بصيغة `YYYY-MM-DD`.
- ☐ **سلّمت** صورة اختبار لكل endpoint (نجاح وخطأ) مع مجلد `backend` (يوم 5)، بدون ملف `.env`.

Phase 2: بناء الـ Front-end مربوط بالـ API — يوم 6 – 9

الهدف: تبنوا الواجهة كاملة، وكل ميزة بتشتغل بـ `fetch` مباشرة مع الـ API اللي بنيتوه بـ Phase 1. **ما في نسخة مؤقتة بالذاكرة هالمرة**: من أول يوم لازم الـ server يكون شغّال عشان تجربوا شغلكم.

شو المطلوب

- **Navbar** و **3 summary cards**: الإجمالي، عدد المصاريف، وأعلى مصروف. الكروت بتحسب من **كل** المصاريف، مش من القائمة المفلترة.
- **Form** لإضافة مصروف: العنوان (مطلوب)، المبلغ (رقم أكبر من صفر)، التصنيف (select)، والتاريخ. مع validation ورسائل خطأ واضحة، وإرسال البيانات بـ `POST` عند النجاح.
- **جدول** أعمدته: العنوان، المبلغ، التصنيف (badge ملوّن)، التاريخ، وزر تعديل وزر حذف.

- **فلترة** حسب التصنيف، مع خيار «الكل».
- **تعديل مصروف** بـ Bootstrap modal معبّى بالبيانات الحالية، والحفظ بيبعت `PUT`.
- **حذف مصروف** بـ `DELETE`.

قواعد الشغل

- كل عملية (عرض، إضافة، تعديل، حذف) بتستخدم `fetch` مع `async/await` من أول لحظة تكتبوها. استخدموا `try/catch`.
- بعد كل إضافة أو تعديل أو حذف، اطلبوا القائمة من جديد من الـ server (`GET`) عشان الصفحة دايمًا تعرض الحقيقة.
- اعرضوا **spinner** Bootstrap أثناء تحميل البيانات.
- أي خطأ اعرضوه بـ **alert** Bootstrap واضح. إذا الـ server طافي، الرسالة تكون مفهومة للمستخدم. إذا رجع `400`، اعرضوا الـ `message` اللي رجعه.
- ما تستخدموا أي مكتبة JavaScript غير Bootstrap.

Checklist: خلصت لما...

- ☐ بضيف مصروف وبيظهر بالجدول (بعد ما يتأكد الحفظ بالـ server).
- ☐ الـ form ما بيقبل حقل فاضي، ولا مبلغ صفر أو سالب.
- ☐ بحذف مصروف وبيختفي من الجدول، وبيتحدث الملخص.
- ☐ الفلترة بالتصنيف بتشتغل، وبقدر أرجع لـ «الكل».
- ☐ بعدّل مصروف من الـ modal وبيتحدث بالجدول والملخص.
- ☐ الـ spinner بيظهر أثناء التحميل، وبيختفي بعده.
- ☐ لما بطفّي الـ server بيطلع alert مفهوم، مش صفحة فاضية أو مكسورة.
- ☐ الصفحة بتظهر منيحة على شاشة الموبايل.

Checkpoint: نهاية يوم 9
تسلّموا المشروع كامل شغال (front-end + backend + database). هاي أول وأهم نقطة تقييم رسمية بالمشروع. يوم 10 مرن لمين لسا ما خلّص.

Phase 3: التحسين والتسليم — يوم 11

الهدف: تخلّوا المشروع جاهز للتسليم، وتضيفوا لمستكم.

مطلوب

- **Responsive:** جربوه على عرض شاشة الموبايل من DevTools.
- **CSS Grid:** استخدموه لعرض الـ summary cards (موضوع جديد، ابحثوا عنه).
- **كود نظيف:** أسماء واضحة، تعليقات عند الحاجة، وما يبقى `console.log` متروك.
- **README:** كيف بنشغّل المشروع من الصفر (إنشاء الـ database وتشغيل `schema.sql`، وكتابة `.env`، وتثبيت الـ packages، وتشغيل الـ server، وفتح الـ front-end)، وصور للتطبيق، وقسم قصير عن أصعب شي واجهتوه وكيف حليتوه.

- **تسليم نظيف:** بدون مجلد `node_modules` وبدون ملف `.env`.

Bonus (اختاروا واحد أو أكتر)

- رسم بياني للمصاريف حسب التصنيف (Chart.js).
- فلترة حسب الشهر، أو بحث بالعنوان.
- ترتيب الجدول بالنقر على عنوان العمود.
- تصدير المصاريف كملف CSV.
- Dark mode.

Checklist: خلصت لما...

- ☐ المشروع بيشتغل من أول مرة بحسب خطوات الـ README.
- ☐ الصفحة بتظهر منيحة على الموبايل والكمبيوتر.
- ☐ بقدر أشرح أي سطر بالمشروع.

كيف تطلبوا مساعدة

- **قاعدة الـ 30 دقيقة:** إذا علقتوا أكتر من 30 دقيقة على نقطة، اسألوا بالـ Discord. ما في داعي تضيعوا يوم كامل على شي بسيط.
- **بما إنه الـ front-end هالمرة مربوط بالـ backend من أول يوم،** أي مشكلة بالـ backend ممكن توقفكم بالكامل. فما تتردوا تسألوا بدري.
- **قبل ما تسألوا:** افتحوا الـ Console (F12) واقرأوا رسالة الخطأ، وجربوا `console.log` لتشوفوا شو بصير.
- مسموح تبحثوا وتستعينوا بمصادر خارجية، بس **كل سطر بتسلموه لازم تقدروا تشرحوه.**

كيف تكتبوا سؤال منيح

شو	ليش
شو بدكم تعملوا؟	عشان نفهم الهدف مش بس الخطأ.
شو الكود اللي كتبتوه؟	صورة أو نص للجزء المعني بس، مش المشروع كامل.
شو اللي صار؟	رسالة الخطأ بالضبط، أو صورة من الشاشة.
شو جربتوا؟	عشان ما نقترح عليكم شي جربتوه.

التسليم والتقييم

شو بتسلموا؟

- **نهاية Phase 0 (يوم 2):** مجلد فيه المهام الثلاث.
- **نهاية Phase 1 (يوم 5):** مجلد `backend` مع صور اختبار كل endpoint.
- **Checkpoint (نهاية يوم 9):** المشروع كامل شغال (front-end + backend + database).
- **التسليم النهائي (يوم 11):** مجلد المشروع كامل كملف zip، **بدون** مجلد `node_modules` و**بدون** ملف `.env`، مع README. وبعدها عرض قصير (demo) للمشروع.

على شو بنقيّم؟

- **الوظائف:** كل ميزة مطلوبة شغالة وبتحقق الـ Checklist.
- **جودة الكود:** تنظيم، أسماء واضحة، وما في كود مكرر أو متروك.
- **الالتزام بالمتطلبات:** الهيكل، وكود الـ backend، وطريقة الـ validation.
- **التعلم الذاتي:** Phase 0، وكيف تعاملتوا مع الأدوات الجديدة، والـ Bonus.
- **الـ Demo:** فهمكم للكود اللي كتبتوه، وهو المقياس الأساسي لأساسياتكم بالـ front-end بهالمشروع.

بالتوفيق!
هاد المشروع فرصة تشوفوا فيها قديش صرتوا تقدروا تبنوا بأنفسكم. اسألوا، جربوا، وما تخافوا من الأخطاء. هي جزء من الشغل.